

Tech Shop Task – Advanced

Tech Shop is a commercial shop for the newest electronic devices (mobile phones, laptops, AirPods, etc.). Customers visit this shop, explore products by category, inquire about prices, and purchase products with desired quantities (not exceeding available stock).

Your challenge: **Design and implement an enterprise-grade, feature-rich, production-quality e-commerce platform** for Tech Shop. This task is intentionally made highly challenging and comprehensive, requiring advanced expertise in architecture, development, security, and DevOps.

Core Requirements

1. Backend Development

- **Microservice Architecture**
 - Decompose the system into at least **five** independently deployable Spring Boot microservices:
 - Product Service
 - Order Service
 - User Service
 - Inventory Service
 - Notification Service (for order status, restock alerts, etc.)
 - Implement **strict separation of bounded contexts** (DDD), including separate databases per service.
 - **API Gateway**
 - Integrate a production-grade API Gateway (e.g., Spring Cloud Gateway or Kong) to route and secure traffic.
 - **Service Communication**
 - Use **asynchronous messaging** (e.g., Kafka, RabbitMQ) for inter-service communication (e.g., order events, stock updates).
 - Demonstrate **event sourcing** for at least one service (e.g., Order Service).
 - **Caching**
 - Integrate Redis for:
 - Frequently accessed data (product categories, etc.)
 - Distributed cache for session management
 - Implement a **cache invalidation strategy**
 - **Authentication and Authorization**
 - Use **Keycloak** for SSO (Single Sign-On) and OAuth2.
 - Integrate social login (GitHub, Facebook, Google) with fine-grained role-based access control.
 - Secure all APIs using JWT tokens, with refresh and revoke support.
 - Implement **multi-tenancy** (support multiple shops/brands).
 - **Payment Integration**
 - Integrate with **one mock payment gateway** (e.g., Stripe, PayPal).
 - Implement a workflow for payment success/failure, order state transitions, and rollback on failure.
 - **Auditing & Logging**
 - Implement a distributed tracing and logging solution (e.g., ELK Stack or Grafana Loki).
 - All critical actions must be auditable with user attribution.
 - **Testing**
 - Achieve at least **85% code coverage** for unit and integration tests (JUnit, Mockito, Testcontainers).
 - Write contract tests for inter-service APIs (e.g., using Pact).
-

2. Frontend Development

- **Framework**
 - Use React (with TypeScript) or Angular.
 - Architect with a **modular, scalable structure** using Redux, React Query, or equivalent.
- **Features**
 - Full-featured UI for product browsing, filtering, cart, checkout, user auth (with SSO).
 - Real-time updates (WebSockets) for order status and stock changes.
 - Comprehensive error handling, optimistic updates, and UI skeletons/loaders.
- **Accessibility & Responsiveness**
 - Fully responsive (mobile, tablet, desktop).
 - Meet **WCAG 2.1 AA** accessibility standards (use axe or Lighthouse for validation).
- **Testing**
 - Achieve at least **80% coverage** with unit tests (Jest, React Testing Library).
 - End-to-end tests with Cypress or Playwright.

3. DevOps & Infrastructure

- **Containerization & Orchestration**
 - Containerize all components (Docker).
 - Write production-ready multi-stage Dockerfiles.
 - Use **Kubernetes** (Minikube or k3d) for orchestration.
 - Create Kubernetes manifests (deployments, statefulsets, services, ingresses, secrets, configmaps, etc.) for all services.
 - Set up a **Kubernetes-native CI/CD pipeline** (e.g., using GitHub Actions or Jenkins X).
 - **Observability**
 - Integrate Prometheus and Grafana for monitoring.
 - Set up alerts for critical failures and performance bottlenecks.
 - **Security**
 - Enforce HTTPS everywhere.
 - Regular vulnerability scans (e.g., Trivy, Snyk) and dependency updates.
 - Implement rate limiting, API throttling, and secure secrets management (e.g., using HashiCorp Vault or Kubernetes Secrets).
-

4. Bonus/Stretch Requirements

- **Scalability**
 - Demonstrate horizontal scaling of stateless services and resilient handling of failures (e.g., circuit breaker, retries).
 - **Data Consistency**
 - Implement eventual consistency across services (e.g., using the Saga pattern for distributed transactions).
 - **Architecture Documentation**
 - Provide:
 - A comprehensive architecture diagram
 - Sequence diagrams for critical flows (e.g., placing an order)
 - API documentation (OpenAPI/Swagger)
 - **Domain-Driven Design (DDD)**
 - Apply DDD principles throughout:
 - Define aggregates, entities, value objects, repositories, and services in code.
 - Explicitly document bounded contexts and anti-corruption layers.
 - **Localization and Internationalization**
 - Support multiple currencies and languages (with dynamic switching in UI).
 - **Performance**
 - Load-test the system (e.g., with k6 or JMeter) and provide a report demonstrating the system can handle at least **2000 concurrent users**.
-

5. Submission

- All source code must be hosted on GitHub.
 - Include:
 - Complete README with setup, architecture, and usage instructions
 - All relevant scripts/configs for building, testing, and deploying the application
 - Sample data and environment files for local development
-

Evaluation Criteria

- **Enterprise-level code quality and structure**
 - **Effective use of best practices (design, security, testing, DevOps)**
 - **Ability to deliver a truly production-ready, scalable, and secure solution**
 - **Depth of technical documentation**
-

Note:

This task is intentionally challenging and mirrors the requirements for architect path. Only proceed if you are highly confident in your abilities to deliver production-grade, enterprise-level systems. Mere implementation of CRUD operations or basic features will not be sufficient to consider this task as completed or accepted.